HOCHSCHULE HAMM-LIPPSTADT (HSHL)

Firmware Build Environment Issues & Resolutions Report

Band01_Rev1 Smartband · Task 3 Development Log

Prepared by: Masrur Jamil Prochchhod
Matriculation: 2210028
Institution: Hochschule Hamm-Lippstadt (HSHL)
Date: 28 May 2026
Environment: CCS v12.8.1 · TI SimpleLink CC2640R2 SDK v5.30.01.11 · Windows 11
Workspace: workspace_v12 (app) · workspace_v13 (stack library)

1. Purpose of This Report

This report documents the firmware build environment issues encountered during the first attempt to compile the Band01_Rev1 smartband firmware in Code Composer Studio on 28 May 2026. It serves as a technical log for the final project presentation and as a permanent reference so that future developers working with this repository do not encounter the same problems.

Each issue is described with its exact error message, root cause analysis, resolution steps, and the files committed to the repository as a result. Additional issues encountered in later sessions will be appended to this document before the final presentation.

Item	Detail
Development Environment	Code Composer Studio v12.8.1
SDK	TI SimpleLink CC2640R2 SDK v5.30.01.11
OS	Windows 11
App Workspace	C:\Users\mjami\workspace_v12 (smart_band_v001_app)
Stack Workspace	C:\Users\mjami\workspace_v13 (stack library build)
Target Device	CC2640R2FRSMR — BLE 5.0 SoC (ARM Cortex-M3)
Build Date	28 May 2026

2. Issues Overview

Three separate build issues were identified and resolved in sequence on 28 May 2026. They are listed below in the order they were encountered and are each documented in full detail in Section 3.

#	Type	Issue Description	Status
1	Missing file	build_config.opt absent from repository — all 25+ source files failed to compile	RESOLVED
2	Missing library	BLE stack .lib file and two linker support files not present — linker could not produce final binary	RESOLVED
3	Header mismatch	gapbondmgr.h declared two functions differently from the compiled stack library — symbol conflict at link time	RESOLVED

3. Detailed Issue Documentation

Issue 1	**Missing build_config.opt — Compiler Options File Absent from Repository** *All 25+ source files failed to compile*	RESOLVED

3.1.1 Error Messages

The following errors appeared for every single source file (25+ files) when attempting to build:

```
Cannot open command file '/TOOLS/build_config.opt': No such file or directory

RFCC26XX_singleMode.c    /smart_band_v001_app/Drivers/RF
simple_peripheral.c      /smart_band_v001_app/Application
i2c_sensors.c            /smart_band_v001_app/Application
[... repeated for all 25+ source files ...]

#35 #error directive: "A required symbol (RF_SINGLEMODE) is missing."
RFCC26XX_singleMode.c    line 64
```

3.1.2 Root Cause

Every source file in the project references a compiler options file through a flag embedded in the CCS project build settings:

```
--cmd-file="${REF_PROJECT_1_LOC}/TOOLS/build_config.opt"
```

The variable **`REF_PROJECT_1_LOC`** was intended to point to the companion stack library project folder (**`smart_band_v001_stack_library`**) which existed in the original developer's workspace. When the repository was created, only the app project was committed to GitHub — the stack project and all files within it, including **`TOOLS/build_config.opt`**, were never pushed.

On any fresh machine, **`REF_PROJECT_1_LOC`** resolves to an empty string, making the path become **`/TOOLS/build_config.opt`** — an absolute path from the root of C: drive that does not exist. The **`RF_SINGLEMODE`** error is a direct consequence: the missing file defines this symbol, and without it the RF driver hits a hard **`#error`** directive on line 64 that stops compilation entirely.

3.1.3 Resolution

The file was created manually at **`C:\TOOLS\build_config.opt`** (matching the resolved absolute path) with the standard preprocessor symbol definitions required for a CC2640R2F BLE4 single-mode peripheral application. The file was then also committed to the repository at **`TOOLS/build_config.opt`** so future developers have it available on clone.

Contents of build_config.opt:

```
-DRF_SINGLEMODE     ; CC2640R2F operates in single-mode BLE (not multi-mode)
-DUSE_ICALL         ; Use TI ICall inter-task messaging layer
-DICALL_LITE        ; Lightweight ICall variant for this application
-DPOWER_SAVING      ; Enable deep sleep between BLE events
-DHEAPMGR_SIZE=0    ; Use dynamic heap sizing
-DMAX_NUM_BLE_CONNS=1 ; Maximum one simultaneous BLE connection
-DMAX_NUM_PDU=5     ; Maximum 5 protocol data units
-DMAX_PDU_SIZE=27   ; Maximum PDU size in bytes
```

File	Action
TOOLS/build_config.opt	Created from scratch — was completely absent from repository. Committed to repo.

Issue 2	Missing BLE Stack Library — Linker Could Not Find Required Binary Files *Compilation succeeded but linking failed*	RESOLVED

3.2.1 Error Messages

After Issue 1 was resolved, compilation succeeded but the linker failed with the following errors:

```
#10008-D cannot find file "/FlashROM_Library/.lib"
#10008-D cannot find file "/FlashROM_Library/ble_r2.symbols"
#10008-D cannot find file "/FlashROM_Library/lib_linker.cmd"
#10192 Failed linktime optimization
gmake[1]: *** [smart_band_v001_app.out] Error 1

symbol "RF_cancelCmd" redeclared with incompatible type
symbol "RF_open" redeclared with incompatible type
[... 15+ similar RF symbol conflicts ...]
```

3.2.2 Root Cause

The smartband linker script (**`ccsObjs.opt`**) references three files at an absolute path:

```
-l"/FlashROM_Library/ble_r2.symbols"
-l"/FlashROM_Library/lib_linker.cmd"
-l"/FlashROM_Library/.lib"
```

These are the compiled output files of the BLE protocol stack — the pre-built binary that the application links against. They are produced by building the companion stack library project. Since that project was never committed to the repository, these files never existed on a fresh machine.

Important correction on path meaning: **`REF_PROJECT_1_LOC`** pointed to the stack project's root folder. The compiled outputs land in a **`FlashROM_Library/`** subdirectory within that project. The linker script hardcodes **`C:\FlashROM_Library\`** as the expected location — meaning the three output files must be placed there explicitly.

First attempt — wrong SDK stack: **`ble5_simple_peripheral_cc2640r2lp_stack_library`** from the BLE5 SDK examples was tried first. This produced 20+ additional symbol conflict errors because the smartband firmware was built against the BLE4 stack (**`blestack`**), not BLE5. The correct project path is:

```
C:\ti\simplelink_cc2640r2_sdk_5_30_01_11\examples\rtos\
CC2640R2_LAUNCHXL\blestack\simple_peripheral\tirtos\ccs\
simple_peripheral_cc2640r2lp_stack_library
```

3.2.3 Resolution

The correct BLE4 stack library project was imported into a separate CCS workspace (workspace_v13) and built successfully. The three output files were then copied to C:\FlashROM_Library\ where the linker expects them:

File	Source (after building stack library)	Destination
.lib	workspace_v13\...\FlashROM_Library\ simple_peripheral_cc2640r2lp_stack_library.lib	C:\FlashROM_Library\.lib
lib_linker.cmd	workspace_v13\...\FlashROM_Library\ lib_linker.cmd	C:\FlashROM_Library\ lib_linker.cmd
ble_r2.symbols	workspace_v13\...\FlashROM_Library\ ble_r2.symbols	C:\FlashROM_Library\ ble_r2.symbols

Important for future developers: These three files are binary and cannot be committed to the repository. Any developer setting up this project on a new machine must build **`simple_peripheral_cc2640r2lp_stack_library`** from the blestack SDK examples and copy its FlashROM_Library output to **`C:\FlashROM_Library\`** before the app project will link successfully.

Issue 3	**Mismatched gapbondmgr.h — Header File Out of Sync with Stack Library** *Two GAPBondMgr functions redeclared with incompatible types*	RESOLVED

3.3.1 Error Messages

After Issues 1 and 2 were resolved, two final linker errors remained:

```
symbol "GAPBondMgr_PasscodeRsp" redeclared with incompatible type:
smart_band_v001_app

symbol "GAPBondMgr_Register" redeclared with incompatible type:
smart_band_v001_app

#10192 Failed linktime optimization
gmake[1]: *** [smart_band_v001_app.out] Error 1
```

3.3.2 Root Cause

The repository contained its own local copy of **`gapbondmgr.h`** in the **`Include/`** folder. This header file declares the function signatures for **`GAPBondMgr_PasscodeRsp`** and **`GAPBondMgr_Register`**. The local copy was an older version that declared these two functions with different parameter types than the versions that actually exist in the compiled stack library built from the current SDK.

This is a classic header/library version mismatch. The header file describes what functions are available and how to call them. The compiled .lib file contains the actual machine code. When these two descriptions differ, the linker detects an incompatible type redeclaration and refuses to link.

The **`fc`** (file compare) command confirmed the difference — the SDK's current blestack version of **`gapbondmgr.h`** contained additional functions and updated declarations not present in the repository's copy.

3.3.3 Resolution

The repository's **`Include/gapbondmgr.h`** was replaced with the correct version from the SDK:

```
Source:      C:\ti\simplelink_cc2640r2_sdk_5_30_01_11\source\ti\
             blestack\profiles\roles\gapbondmgr.h

Destination: C:\ti\smart_band_v001_app\smart_band_v001_app\
             Include\gapbondmgr.h

Command:     copy /Y "...\blestack\profiles\roles\gapbondmgr.h"
                     "...\Include\gapbondmgr.h"
```

File	Action
Include/gapbondmgr.h	Replaced with correct SDK blestack version to match the compiled stack library. Committed to repository.

4. Final Build Result

After resolving all three issues, **`smart_band_v001_app`** compiled and linked successfully on 28 May 2026 with **0 errors**. The following output files were generated:

Output File	Format	Description
smart_band_v001_app.out	ELF binary	Used by CCS debugger for flashing and step-through debugging with JTAG probe
smart_band_v001_app.hex	Intel HEX	Used for direct flashing tools — stored in repository from original build
smart_band_v001_app.map	Linker map	Shows exact flash and RAM memory usage for every module

Build status: 0 errors · Build completed successfully on 28 May 2026 · Output confirmed in FlashROM_StackLibrary folder

Firmware state: The firmware compiles and links cleanly. As documented in Task 2, live sensor reading calls are currently commented out — the firmware transmits a hardcoded placeholder value (**`0x7788`**) via BLE GATT. It is ready to flash once a JTAG debug probe and power supply are connected.

5. Summary for Final Presentation

- The firmware repository was missing two essential text files (build_config.opt and an updated gapbondmgr.h) and one binary dependency (the compiled BLE4 stack library) that are required to build the project on any machine other than the original developer's.
- All three issues were identified, root-caused, and resolved. The two text files were committed back to the repository so the project is now buildable from a fresh clone.
- The BLE stack library must be built separately from the TI SDK blestack examples — this is a standard TI two-project architecture that was not documented in the original repository. A note has been added explaining this requirement.
- The firmware itself is correct and complete — once built, it transmits placeholder sensor data (`0x7788`) via BLE GATT and is ready for live sensor activation once hardware is available.
- Additional build or runtime issues encountered during hardware testing will be documented here and presented as part of the complete development log at the final presentation.

6. References

Texas Instruments. (2020). SimpleLink CC2640R2 SDK BLE-Stack User Guide. Texas Instruments.

Texas Instruments. (2018). BLE5-Stack User's Guide for CC2640R2F. Texas Instruments.

Texas Instruments. (2019). CC2640R2F SimpleLink Bluetooth Low Energy Wireless MCU Datasheet (SWRS204). Texas Instruments.

Prochchhod, M. J. (2026). PCB Component Analysis Report — Band01_Rev1, Task 1. HSHL.

Prochchhod, M. J. (2026). Firmware Architecture & Code Analysis Report — Band01_Rev1, Task 2. HSHL.